Image Convolution with Filters: Performance Optimization through CUDA Parallelization

Marco De Stefano
Department of Information Engineering
University of Florence
marco.destefano@edu.unifi.it

Abstract

This project investigates the performance characteristics of image convolution filtering across different data layouts and computational platforms. We compare sequential CPU implementations using Array-of-Structures (AoS) and Structure-of-Arrays (SoA) layouts against GPU-accelerated CUDA implementations. Our analysis evaluates how data organization affects memory access patterns and computational efficiency at different image scales. Additionally, we examine GPU-specific optimizations including tiling strategies and block dimension tuning to identify performance bottlenecks and opportunities for acceleration. The results demonstrate the interplay between data layout design, memory hierarchy utilization, and GPU architectural constraints, providing insights into effective strategies for optimizing image processing workloads on heterogeneous computing platforms.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

Image convolution is one of the most important operations in computer vision and digital signal processing. It allows us to process images by applying different filters in order to enhance or extract specific information.
Some common examples of filters are:

- **Gaussian:** performs a smoothing operation on the image.
- **Sharpening:** enhances the edges of the image.
- **Edge Detection:** highlights the edges present in the image.
- **Blurring:** reduces image details by averaging neighboring pixels.

Convolution is a fundamental operation in Convolutional Neural Networks (CNNs), and it has traditionally been used in computer vision as a preprocessing step before extracting characteristic features such as SIFT.
For large images, convolution can be computationally expensive because the overall computational cost of the convolution is:

$$\Theta(H \cdot W \cdot K^2)$$

To improve the execution time of this operation, we exploit the CUDA framework to parallelize the computation on a GPU. In this work, we investigate the following aspects:

- **Sequential Baseline**: implementation and performance evaluation of the CPU-based version.
- **GPU Acceleration**: exploitation of CUDA massive parallelism to speed up the convolution.
- **Scalability Analysis**: an in-depth study of the performance behavior with respect to kernel size, image size, and available computational resources.

1.1. Contributions

This work presents a systematic framework for analyzing image convolution performance on CPU and GPU platforms, focusing on optimization strategies and reproducible benchmarking.

1. **Dual CPU/GPU Implementations:** Sequential C++ implementations (AoS and SoA layouts) and CUDA-based GPU versions (non-tiled and tiled kernels), enabling direct comparison between architectural acceleration and memory layout effects.

2. **Systematic Benchmarking:** Extensive experiments varying image size, kernel size, and thread block dimensions, with 100 iterations per configuration and CUDA event synchronization for accurate timing. Results are exported in CSV format for reproducibility.

3. **Optimization Analysis:** Evaluation of data layout (AoS vs. SoA), shared memory tiling, block dimension tuning, and GPU warmup strategies to assess their impact on memory efficiency, occupancy, and overall performance.

4. **Automated Analysis Pipeline:** A Python-based tool generating five performance figures (CPU vs. GPU comparison, speedup analysis, layout trade-offs, tiling efficiency, and block optimization), along with summary statistics and optimal configuration identification.

5. **Reproducible Experimental Design:** Modular C++ benchmark code with configurable kernels, command-line interface support, and structured result storage for scalable and repeatable experimentation.

Overall, the framework highlights how combined optimization strategies lead to substantial performance gains in GPU-accelerated image processing.

2. Algorithm Description

2.1. Mathematical Definition

This section will explain the algorithm of the convolution to be clear how can it be parallelize. In the sequential code, the input image is been processed with the kernel filter, elaborating a pixel one by one.
If the pixel i is the one, and we have a kernel filter with size 3x3 then the operation for one channel is:

$$p_{i,j} = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \hat{I}(i+m, j+n) \cdot K(m,n)$$

where $\hat{I}$ denotes the image extended with padding. In general, when a neighboring pixel falls outside the image boundaries, different padding strategies can be adopted:

- **Zero Padding:** pixels outside the image domain are assigned a value of zero.
- **Boundary Clipping:** pixels outside the image domain are assigned the value of the nearest valid pixel inside the image.
- **Reflection:** pixels outside the image domain are obtained by mirroring the image with respect to its boundaries.
- **Wrap Around:** the image is treated as periodic, and out-of-bound pixels are mapped using modular indexing.

In this work, we adopt the **Boundary Clipping** strategy. Formally, given an image of width W and height H, the clipped coordinates are defined as:

$$x' = \min(\max(x, 0), W-1)$$

$$y' = \min(\max(y, 0), H-1)$$

Therefore, the pixel value used in the convolution becomes:

$$I_{\text{clip}}(x, y) = I(x', y')$$

3. Computational Complexity

For an image of size $W \times H$ and a square kernel of size $K \times K$, each output pixel is computed as the sum of the products between the kernel values and the corresponding image pixels. Specifically, this requires K^2 floating-point multiplications and $K^2 - 1$ floating-point additions, for a total of approximately $2K^2$ floating-point operations (FLOPs) per output pixel.

Considering an RGB image, with three color channels, and an output of size $H \times W$, the overall computational complexity of the 2D convolution operation can be expressed as:

$$O(3 \cdot H \cdot W \cdot K^2)$$

3.1. Kernel Filter Types

In both the operation, sequential and parallel, i have created a function that calculate the filter of a various size. In the sequential part, the results saved are only with the filter Gaussian because we want to study the performance gained and the filter type has no correlation with the complexity of the convolution moreover because the sequential implementation is used only as a baseline.
Meanwhile, the parallel part has function to create filters of each type:

- **Gaussian:** this filter can make the image more smooth.
- **Edge:** this filter can detect edge of the image.
- **Box:** make the average of the neighbor pixels.
- **Sharper:** this filter can make the image more sharp.

Those filters are used for visualization, otherwise in the experiment it used always the gaussian filters. In the experiment, the benchmark use progressively these dimention tile: $[3, 5, 7, 9, 11, 15, 21, 23]$.

4. CPU Implementation

4.1. Sequential Implementation

As what we have said before, the sequential CPU implementation serves as the baseline for performance comparison with GPU-accelerated variants. In the following file, the program loads an image from the **Berkeley Segmentation Dataset**, which serves only as a template for experimental purposes.

The filter is then normalized upon creation. Since symmetric filters are used, the convolution operation is identical to the correlation operation, as flipping the kernel does not change its values.

Then, all benchmarks are conducted on the same hardware platform using identical image data to ensure fair comparison. Each experiment consists of 50 consecutive iterations of the convolution operation, with timing measurements averaged across all iterations to reduce variance from system noise. All the average times for all data layout are saved in a file csv.

Those benchmark are made changing the resolution and the kernel, to print the complexity behavior of the convolution operation.

Also, we compare two different data layouts: **Array of Structures (AoS)** and **Structure of Arrays (SoA)**.
In the AoS layout, each pixel is stored as a structure containing its three color channels. During convolution, the algorithm iterates over each pixel, accesses the neighboring pixels through the array of structures, multiplies each channel by the corresponding kernel coefficient, sums the results, and writes the output in a new AoS array.

In the SoA layout, each color channel is stored in a separate contiguous array. During convolution, the algorithm iterates over each pixel, but accesses the three color channels independently from three separate arrays.

5. GPU Parallel Implementation

In this project, we implemented a GPU-based version of the convolution algorithm in order to exploit the computational capabilities of modern GPUs. To evaluate the potential benefits of GPU acceleration, we developed two different parallel implementations:

- **Non-Tiled Implementation:** each thread, identified by its thread and block indices, computes a single output pixel. The thread iterates over the kernel window, performs the required multiplications and accumulations, and stores the result in the corresponding output location. In this version, all image data are accessed directly from global memory, which is characterized by high latency.
- **Tiled Implementation:** at the beginning of the kernel execution, each block loads a tile of the image into shared memory, including a halo region required for convolution. This allows threads within the same block to reuse data without repeatedly accessing global memory. In our implementation, the tile dimension is determined at runtime, and an additional halo of one pixel is included. The execution is therefore divided into two phases: a loading phase and a computation phase.

In addition, we implemented the convolution using both AoS (Array of Structures) and SoA (Structure of Arrays) data layouts to evaluate the impact of memory organization on global memory access patterns. In particular, this comparison highlights the effect of coalesced memory accesses, which are crucial for maximizing memory throughput on GPUs.

5.1. CUDA Kernel Organization

As what we said, the GPU convolution is organized through a spatial tiling strategy and without. The output image is partitioned into square tiles of size $X \times X$ pixels. Each CUDA block is responsable for computing one output tile and uses additional threads to load into shared memory the pixels required to handle kernel borders.

5.2. Shared Memory Usage

In this implementation, the shared memory is allocated dynamically based on the tile size. For instance, if a tile contains X total pixels (including the required boundary halos for the convolution filter), the allocated shared memory corresponds to 3X elements.

To comprehensively analyze memory access patterns and hardware performance, two distinct kernel versions were implemented, exploring different shared memory layouts:

- **Array of Structures (AoS):** In this kernel, the Red, Green, and Blue channels of each pixel are stored contiguously in memory. This layout maximizes spatial locality, which is highly beneficial since each thread needs to fetch all three channels simultaneously during the convolution operations.
- **Structure of Arrays (SoA):** In the alternative kernel, the color channels are segregated into three distinct, contiguous arrays (one for Red, one for Green, one for Blue). This layout is traditionally employed to favor memory coalescing and minimize shared memory bank conflicts across threads in the same warp.

For both versions, the execution begins with a collaborative loading phase, where threads cooperatively fetch pixels from the global memory into the shared memory tile. Once the entire tile is loaded and the threads are synchronized, all threads can access their required neighborhood data directly from the ultra-fast shared memory, effectively bypassing the slower global memory during the intensive computation phase.

5.3. Constant Memory Usage

We use the constant memory to save the filters because they didn't change value during the execution and the constant memory is perfect for read only access.

Second, and most importantly, constant memory supports a hardware-level broadcast mechanism. During the convolution loop, all threads within a single warp access the exact same element of the filter array simultaneously. Instead of performing 32 individual memory transactions, the constant cache reads the value once and broadcasts it to all 32 threads in a single cycle. This drastically reduces memory bandwidth consumption and accelerates the arithmetic operations.

Due to the inherent architectural constraints of CUDA, which require constant memory to be statically sized at compile time, the filter array is allocated with a maximum predefined dimension.

5.4. Single-Pass RGB Convolution

In this implementation, the convolution filter is applied to all three color channels (Red, Green, and Blue) simultaneously within a single kernel launch.

Instead of processing each channel in separate passes, each thread is responsible for computing the final RGB values of its assigned pixel at once.

This architectural choice is driven by the **memory-bound** nature of image convolution. By processing the three channels concurrently, the kernel fetches the necessary pixel data from global (or shared) memory only once.

This single-pass approach maximizes data locality and drastically reduces redundant global memory transactions compared to a decoupled, per-channel execution model, ultimately leading to higher memory bandwidth utilization.

Furthermore, this architecture completely eliminates the **Kernel Launch Overhead** associated with dispatching separate tasks for each channel. By issuing a single, unified command to the GPU, the hardware remains constantly saturated, preventing synchronization delays and idle times that would otherwise occur between multiple kernel invocations.

6. Experimental Conditions

Hardware All benchmarks were executed on a single workstation with the following specifications:

- **CPU:** AMD Ryzen 9 9900X (12 Cores, 24 Threads)
- **RAM:** 32 GB DDR5-6000
- **GPU:** NVIDIA GeForce RTX 5070 Ti, 16 GB VRAM

Property	Value
GPU Architecture	
GPU Name	NVIDIA GeForce RTX 5070 Ti
Compute Capability	12.0 (Blackwell)
Number of SMs	70
Max Threads per SM	1536
Max Threads per Block	1024
Max Registers per Block	65536
Memory Specifications	
Total Global Memory	16302 MB ($\approx$ 16 GB)
L2 Cache	49152 KB ($\approx$ 48 MB)
Shared Memory MAX per Block	48 KB
Shared Memory MAX per SM	100 KB
Execution Parameters	
Warp Size	32 threads
Max Grid Dimension	(2×10^9, 65535, 65535)

Table 1. Hardware specifications extracted via CUDA Runtime API (`cudaGetDeviceProperties`).

Software & Environment

- **Host OS:** Windows 11 running Windows Subsystem for Linux (WSL2)
- **Python Environment:** Python 3.13.5 (utilizing some libraries as matplotlib, pandas, numpy, seaborn, pathlib)
- **CUDA Toolkit:** 12.8 (NVCC Compiler)
- **Compilation Flags:** Host code was compiled in **Release** mode, heavily utilizing compiler optimization flags:
  - `-O3`: Aggressive loop unrolling, function inlining, and memory access optimizations.

- `-march=native`: Unlocking the specific modern vector instruction sets (AVX-512) native to the AMD Ryzen 9 9900X processor.

Measurement Methodology To ensure statistical significance and eliminate anomalies in the profiling data:

- A **warm-up kernel execution** was performed prior to any measurement to account for the CUDA driver initialization and context creation overhead.
- To accurately measure parallel execution, `cudaDeviceSynchronize()` was explicitly called before stopping the timer, ensuring all GPU threads completed their workloads.
- The reported execution times represent the average of **50 in sequential and 100 in parallel** consecutive runs for each configuration to mitigate OS scheduling noise.

7. Results and Analysis of Experiments

In this section, we present the execution times and speedups achieved by transitioning the convolution algorithm from a sequential C++ implementation to a parallel CUDA implementation. The primary objective was to ensure a fair comparison by optimizing the sequential baseline to its architectural limits on the CPU, thereby highlighting the actual hardware advantage provided by the GPU. We avoid testing every possible combination of block sizes, tiles, and dimensions, using a bottleneck-driven analysis conduction. All tests were performed on a fixed image from the Berkeley Segmentation Dataset, focusing on the following aspects:

1. Analysis of Convolution Complexity
2. Sequential vs. Parallel Speedup
3. Data Layout Comparison
4. Tiling Impact
5. Block Dimension Optimization

The base image used for all experiments is shown below:

Figure 1. A funny beaver

The parallel application supports three execution modes via command-line arguments:

- **profile**: Used to run a heavy workload specifically designed for external profiling tools (e.g., Nsight Compute).
- **visualize**: Applies the convolution and displays the output image. An optional second argument specifies the filter type:
  - **gaussian**: for smooth blurring and noise reduction.
  - **box**: for a simple, uniform average blur.
  - **edge**: for edge detection and highlighting structural boundaries.
  - **sharper**(default): for enhancing details and edge contrast.
- **benchmark** (default): Executes the full suite of benchmarking experiments.

7.1. Analysis of Convolution Complexity

This section presents the results of the first three experiments. We begin by analyzing the overall computational complexity and the execution times of the optimized sequential CPU implementation versus the parallel CUDA approach.

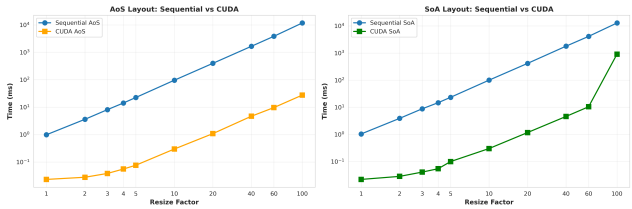

Figure 2. Execution Time vs. Image Resolution: The parallel CUDA implementation drastically outperforms the sequential CPU baseline across all configurations.

As shown in Figure 2, the sequential execution time grows linearly with the number of pixels, whereas the GPU handles the workload with significantly lower latencies. To clarify, these benchmarks were conducted using a fixed 3×3 filter size while progressively scaling the image resolution.

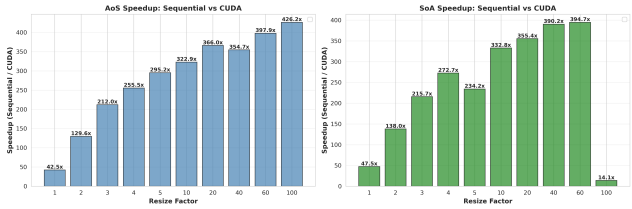

Figure 3. Speedup Analysis vs. Image Resolution.

This clear superiority is quantified in detail in the speedup analysis (Figure 3). For resize factors ranging from 2 to 40, the acceleration provided by the GPU ranges from approximately $30\times$ to over $400\times$ against the fully optimized CPU baseline. The lower speedup observed at Factor 1 is not due to computational inefficiency, but rather to the **kernel launch overhead**. For extremely small images, the baseline time required by the CUDA driver to schedule and launch the kernel becomes the dominant bottleneck, overshadowing the actual computation time.

The theoretical computational complexity of 2D convolution is $\mathcal{O}(H \cdot W \cdot K^2)$. While the cost scales linearly with the number of pixels ($H \cdot W$), it exhibits a strictly quadratic growth with respect to the kernel dimension (K). To isolate and observe this quadratic behavior, the next experiment evaluates execution times using a fixed high-resolution image (Factor $20\times$) while progressively increasing the kernel size.

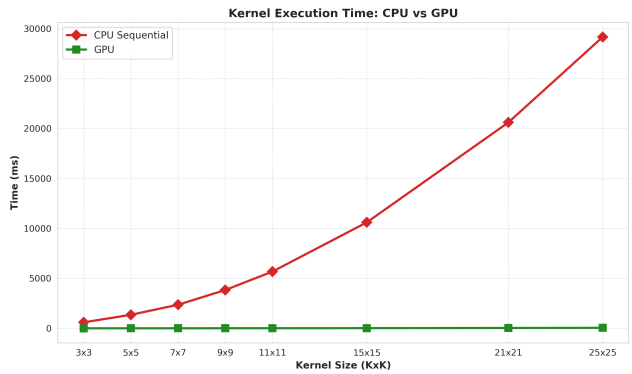

Figure 4. Execution Time vs. Kernel Size. The CPU exhibits a distinct quadratic growth, whereas the GPU execution time scales minimally.

As illustrated in Figure 4, the CPU execution time strictly follows the expected quadratic curve. Conversely, the GPU execution time appears almost constant on this scale, effectively masking the $\mathcal{O}(K^2)$ complexity growth thanks to the massive parallelization of the arithmetic operations.

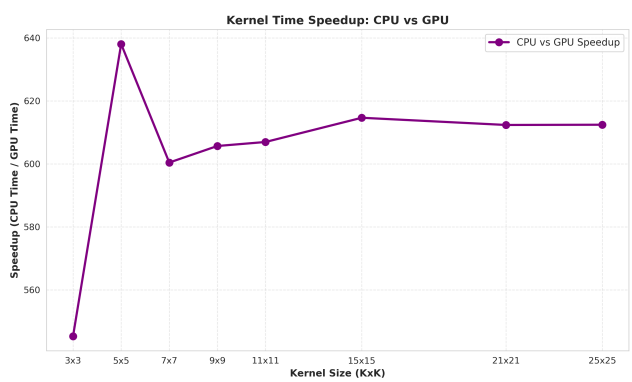

Figure 5. Speedup Scaling with Kernel Size.

This phenomenon is explicitly quantified in Figure 5, which plots the speedup as a function of the kernel size. Compared to the resolution scaling analysis, increasing the filter size yields an even more dramatic performance gap, achieving a speedup of over $600\times$ for the largest tested kernel. This highlights the transition from a memory-bound regime (small kernels) to a heavily compute-bound regime (large kernels), where the GPU's superior arithmetic throughput is fully exploited.

7.2. Analysis with different Data Layout

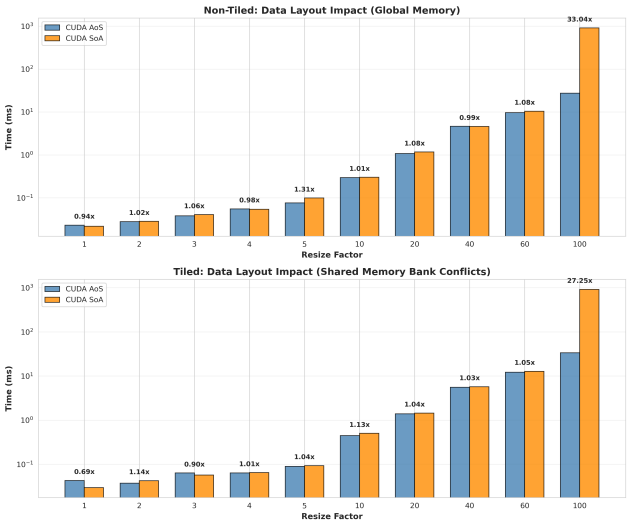

Figure 6. Layout Impact

Figure 6 illustrates the performance comparison between the Array of Structures (AoS) and Structure of Arrays (SoA) layouts. Up to Factor 60, the performance of AoS and SoA is remarkably similar, with minimal fluctuations. However, a massive performance degradation occurs for SoA at Factor 100, where it becomes up to $\approx 30\times$ slower than AoS.
To understand this behavior, we must analyze how GPU memory controllers and warps (groups of 32 threads) interact with the data:

Performance Parity (Up to Factor 60) Generally, SoA is preferred on GPUs because it guarantees perfect *memory coalescing*. Since the R, G, and B channels are stored in separate arrays, consecutive threads in a warp access contiguous memory addresses, maximizing bandwidth. Conversely, AoS stores pixels as **interleaved** channels (RGB-RGB-RGB), which introduces a stride in memory accesses. However, because our convolution kernel requires all three channels simultaneously to compute the final pixel value, the GPU's L1/L2 caches easily fetch the interleaved data in 128-byte cache lines. This effectively mitigates the strided access penalty, allowing AoS to match SoA's performance.

The SoA Collapse (Factor 100) At Factor 100, the image size expands to several Gigabytes. In the SoA layout, the R, G, and B arrays are allocated vast distances apart in Global Memory. When the kernel attempts to fetch the three channels simultaneously, the GPU is forced to access widely separated memory regions concurrently. This leads to severe **TLB (Translation Lookaside Buffer) thrashing** and heavy cache eviction rates. The memory controller becomes bottlenecked trying to keep these distant memory pages active. AoS, on the other hand, keeps the R, G, and B components of each pixel perfectly localized, requiring only a single memory region to be accessed at a time, thereby surviving the extreme scale of the dataset.

	Sequential (ms)		Parallel CUDA (ms)		Speed up	
Factor	**AoS**	**SoA**	**AoS**	**SoA**	**AoS**	**SoA**
1	0.983	1.037	0.023	0.022	42×	47×
2	3.597	3.904	0.028	0.028	130×	138×
3	8.104	8.777	0.038	0.041	212×	216×
4	14.164	14.766	0.055	0.054	256×	273×
5	22.479	23.308	0.076	0.100	295×	234×
10	95.832	100.097	0.297	0.301	323×	333×
20	395.708	415.797	1.081	1.170	366×	355×
40	1652.210	1795.410	4.658	4.601	355×	390×
60	3834.250	4121.340	9.635	10.443	398×	395×
100	11723.000	12852.400	27.509	908.915	426×	14×

Table 2. Execution times and speedups for Sequential vs Parallel CUDA (No Tiled).

To make the other experiments, we would use the AoS data Layout because with this implementation is more robust.

7.3. Block Optimization

For the next result, we first need to define what a block is in CUDA. If a warp is a group of 32 threads that execute the kernel function in parallel, a block is a group of warps.
All the threads within a block can:

- access shared memory.
- synchronize with each other;
- communicate with each other.

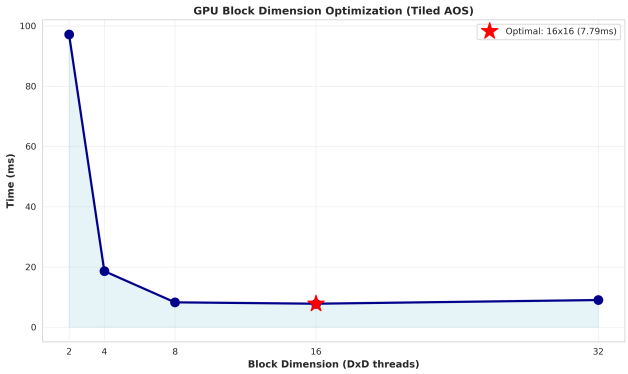

Figure 7. Block Optimization

We can divide the graph into three regions:

- **Underutilization (2×2 – 4×4):** In the left part we can observe a very high execution time. This happens because defining a block of 2×2 means using only four threads. As a result, each block processes only a patch of 2×2 pixels, which leads to poor GPU utilization.

- **Sweet Spot (8×8 – 16×16):** Here the number of threads is higher and provides the scheduler with a sufficient number of warps. This configuration achieves the best execution time.

- **Extreme Usage (32×32):**Utilizing the maximum allowed block size (1024 threads) leads to performance degradation. Based on our hardware specifications, a single SM can host up to 1536 threads and provides a maximum of 65536 registers. With a block size of 1024 threads, the limit of 65536 registers restricts each thread to a maximum of 64 registers ($1024 \times 64 = 65536$). This hardware saturation forces the SM to schedule only **one block at a time**, leaving 512 thread slots completely idle and capping the theoretical occupancy at 66% ($1024/1536$). Furthermore, if the convolution kernel requires even a single register more than 64, the compiler is forced to perform *register spilling*—moving temporary variables into the much slower local memory, which severely bottlenecks the execution speed.

In the next experiment, we will use the best configuration obtained from these results.

7.4. Tiling Effect

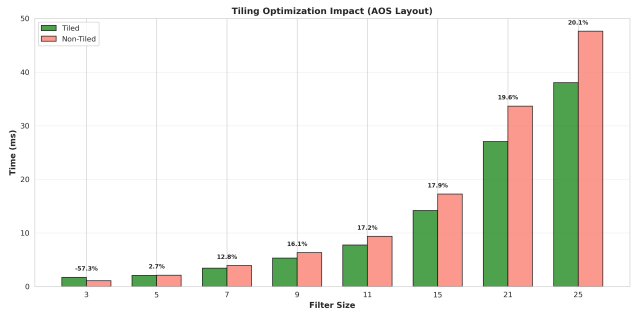

Figure 8. Tiling Effect

This plot compares the execution times of the convolution algorithm using two variants: the **non-tiled implementation** and the **tiled implementation**.
The tiled implementation uses shared memory, whose usage is directly related to the size of the tile. The test is conducted by increasing the tile dimension from 3×3 to 25×25.
The test to plot this graph uses:

- the Array of Structure Layout
- an image input with resolution 9620×6420
- the block dimention optimized with the previous experiment $\Rightarrow 16 \times 16$

Shared Memory Overhead With small tile dimensions we observe worse execution times compared to larger tiles. This happens because the cost of transferring data to shared memory, handling the margins, and performing thread synchronizations does not compensate for the fixed overhead of these infrastructural operations.

Optimization with Tiles Starting from the 5×5 tile size, the computational workload increases and the tiled implementation begins to outperform the non-tiled implementation. This happens because, in the non-tiled kernel, there are many redundant

memory accesses, whereas in the tiled version the entire block is first loaded into shared memory. Once loaded, all threads in the block can reuse this data for their computations.

	Execution Time (ms)		
Filter Size	**AoS Tiled**	**AoS Non-Tiled**	**Improvement**
3	1.732	1.101	-57.3%
5	2.068	2.125	$+2.7\%$
7	3.432	3.934	$+12.8\%$
9	5.317	6.334	$+16.1\%$
11	7.749	9.363	$+17.2\%$
15	14.181	17.263	$+17.9\%$
21	27.061	33.666	$+19.6\%$
25	38.041	47.633	$+20.1\%$

Table 3. Tiling performance impact based on convolution filter size using the AoS layout.

As we can see the improvement of the tiling tecnique increases with the increase of the kernel size, that means if we have use a big filter it is better to use a parallel implementation otherwise the execution time will increase quadratically.
Turning to the Figure 5, we could see, in the plot below, that there is a increment of the speed up until almost $\times 800$.

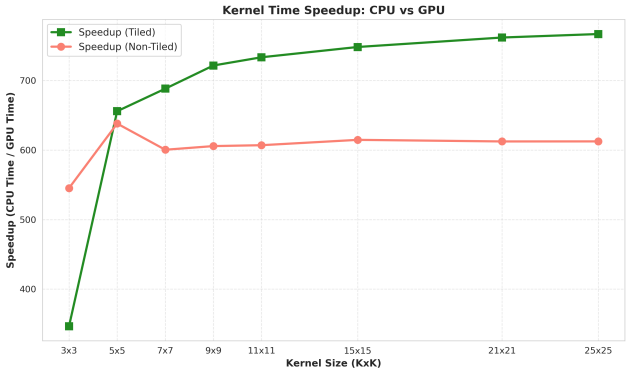

Figure 9. Speed up of the parallelization with and without tiling

Before the conclusions we could analyzes the MAC operations in this implementation.

8. Optimal Configuration Selection

After conducting these preliminary experiments, we used the NVIDIA Nsight Compute profiler to calculate the actual **Throughput** and **Memory Bandwidth** of our best configurations.

Throughput refers to the raw computing power extracted from the hardware during the execution of a kernel in 'profile' mode. Since the core of the algorithm consists of multiplying the value of a pixel by the corresponding spatial weight of the filter and accumulating the result, the best reference metric is not given by individual instructions, but by **MAC (Multiply-Accumulate) operations**.
These results are extracted from the profiling of our implementation on an extreme stress test: a convolution with a tile size of 11×11 on a scaled image of 19240×12840 pixels.
The dimension of a block is 16×16 and the **grid** is allocated dynamically (in this case, the grid is composed of 1203×803 blocks). Overall, 19248×12848 threads are launched, mapping exactly one thread per pixel. According to the profiler reports, the theoretical occupancy is 100%.
To calculate the **Throughput**, we use the following formula:

$$\text{GigaMACs/s} = \frac{W \times H \times C \times K^2}{t_{\text{kernel}} \times 10^9} = \frac{\text{Total MACs}}{t_{\text{kernel}} \times 10^9}$$

To calculate the **latency of MACs operations** the formula is $1000/\text{GMAC/s} = \text{ps/MAC}$. So, we can see the plot below that represents the time to execute a MAC operation.

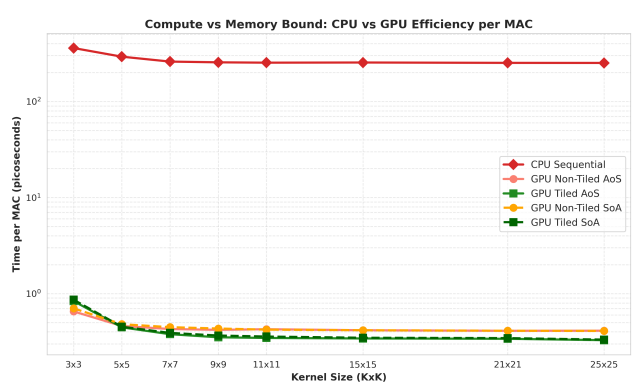

Figure 10. Latency of a MAC operation

Meanwhile, the **Memory Bandwidth** is a crucial indicator for evaluating the quality of the memory access patterns implemented in the code. Optimizations such as coalesced accesses or the targeted use of Shared Memory serve precisely

to maximize the use of the available bandwidth, preventing global memory latency from becoming the main bottleneck. The effective bandwidth is calculated as:

$$MB = \frac{\text{Data}_{\text{R/W}}}{t_{\text{kernel}}}$$

Using these formulas on the times obtained in the report from the profiler, we have the following results:

Kernel	Execution Time (ms)	Throughput (GigaMACs/s)	Memory Bandwidth (GB/s)
SoA - Base (No Tile)	35,208	2548,62	42,13
AoS - Base (No Tile)	32,849	2731,65	45,15
SoA - Tiled	30,094	2981,72	49,28
AoS - Tiled (Best)	**29,517**	**3040,01**	**50,25**

After these results we can confirm analytically that is better using a AoS layout with tiling with a 16×16 dimention block to have a better performance.

9. Performance Analysis Summary

The primary objective of this project was to accelerate a convolution-based image processing algorithm using CUDA. By comparing a parallel implementation against a heavily optimized, vectorized sequential baseline (AVX-512), the experimental evaluation revealed crucial insights into GPU hardware behavior at scale. The main findings are summarized as follows:

- **High-Efficiency GPU Acceleration:** The parallel implementation drastically outperformed the optimized sequential baseline, achieving peak speedups of over **400**$\times$ during heavy workloads - without using Tiling.
- **Optimal Hardware Occupancy:** Empirical tuning demonstrated that a 16×16 thread block size represents the absolute "sweet spot" for the Blackwell architecture. This configuration perfectly balances resource utilization (registers and warps) and latency hiding, maximizing occupancy without exceeding the hardware limits of the Streaming Multiprocessors.
- **Tiling and Shared Memory Scaling:** The adoption of Shared Memory Tiling proved to be highly effective as computational intensity increased. While synchronization overhead dominated for small 3×3 filters, the tiled approach provided a significant performance boost of up to **20.1%** for 25×25 filters. This confirms that tiling is a superior strategy for large-kernel convolutions by successfully minimizing redundant Global Memory transactions.
- **Memory Layout Robustness:** Testing layouts at extreme scaling factors challenged the traditional assumption of SoA superiority. While SoA and AoS performed similarly at standard scales due to cache line efficiency, the SoA layout suffered a catastrophic performance collapse (becoming **30**$\times$ **slower**) at Factor 100 due to massive TLB thrashing. Conversely, the AoS layout demonstrated remarkable robustness, maintaining locality and predictable performance even at gigapixel scales.

Ultimately, the combination of optimal block sizing, strategic use of shared memory for large kernels, and the robust AoS memory layout resulted in a highly scalable architecture, capable of processing massive datasets with near-optimal hardware utilization.

10. Future Work

In this project, we designed a single-pass CUDA kernel that computes the convolution for all three color channels (Red, Green, and Blue) simultaneously. A highly promising avenue for future work is decoupling this process by launching independent kernels—or utilizing a single, channel-agnostic kernel invoked three times—for each respective channel.

Transitioning to a single-channel processing model could unlock several advanced optimizations:

- **Register Pressure Reduction:** By processing only one channel per thread, the num-

ber of hardware registers required by the kernel drops significantly. Lower register usage directly translates to higher theoretical occupancy, allowing the GPU scheduler to maintain more active warps concurrently on each Streaming Multiprocessor (SM).

- **Concurrent Execution via CUDA Streams:** The independent computations for the R, G, and B channels could be dispatched asynchronously using separate CUDA Streams. This would allow the hardware to overlap the execution of the three channels concurrently, maximizing resource utilization.

- **Re-evaluating the SoA Layout:** As observed in our data layout analysis, the Structure of Arrays (SoA) layout suffered a massive performance collapse at extreme scales (Factor 100) due to TLB thrashing, since the single kernel had to fetch from three distant memory regions simultaneously. A single-channel kernel would completely eliminate this bottleneck. Each thread block would access only a single, perfectly contiguous array at a time, potentially allowing the SoA layout to achieve perfect memory coalescing and finally outperform AoS at massive scales.

11. References

References

[1] Kirk, D. B., & Hwu, W. M. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann.

[2] NVIDIA Corporation. (2023). *CUDA C++ Programming Guide*. Version 12.2.

[3] NVIDIA Corporation. (2023). *CUDA C++ Best Practices Guide*. Version 12.2.